

Monte_Carlo_Sim

1. Scientific Question

The question is: “How do different hyperparameter combinations in LightGBM affect the model’s ability to generalize when trained on data with complexity similar to real-world salary prediction problems?”

This Monte Carlo simulation is motivated by the observation that the LightGBM model from Project III achieved low RMSE on both training and validation datasets. While this suggests strong model performance, it raises concerns about potential overfitting, especially when applied to new, unseen data.

Rather than comparing entirely different model types, which can be computationally intensive, this simulation focuses on the impact of tuning LightGBM’s hyperparameters. By repeatedly training and evaluating the model on synthetic datasets that mirror the complexity and structure of the original salary data, this experiment quantifies how hyperparameter choices influence generalization performance and predictive stability across multiple runs. The goal is to identify combinations that consistently yield low error, rather than those that perform well by chance on a single dataset.

2. Data

The simulation generates synthetic salary data based on a linear combination of job-related and demographic features. The model includes both numeric and categorical variables, with categorical variables encoded as numeric factors.

Let:

- ($i = 1, 2, \dots, n$) denote individual employees.
- (Y_i): salary of individual (i).
- (J_i): average salary associated with the job title of individual (i)
- ($E_i \in 1, 2, 3, 4$): numeric encoding of experience level (EN = 1, MI = 2, SE = 3, EX = 4).
- ($C_i \in 1, 2, 3$): numeric encoding of company size (S = 1, M = 2, L = 3).

- ($T_i \in 1, 2, 3, 4$): numeric encoding of employment type (FL = 1, CT = 2, PT = 3, FT = 4).
- ($R_i \in [0, 50, 100]$): remote work ratio (percent).
- ($\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$): random noise.

The salary for each individual is generated by the model:

$$Y_i = \beta_0 + \beta_1 J_i + \beta_2 E_i + \beta_3 C_i + \beta_4 T_i + \beta_5 \left(\frac{R_i}{100} \right) + \varepsilon_i$$

Where:

- ($\beta_0 = 0$): intercept.
- ($\beta_1 = 0.7$): weight for job title average salary.
- ($\beta_2 = 3000$): effect per level of experience.
- ($\beta_3 = 1500$): effect per company size level.
- ($\beta_4 = 1000$): effect per employment type level.
- ($\beta_5 = 500$): effect for full remote work.
- ($\varepsilon_i \sim \mathcal{N}(0, 10000^2)$): Gaussian error with standard deviation 10,000.

Assumptions

1. **Linearity:** The relationship between predictors and salary is linear.
2. **Additivity:** Effects of job title, experience, company size, employment type, and remote ratio are additive.
3. **Homoscedasticity:** Constant variance of the error term (ε_i).
4. **Normality:** The error term is normally distributed.
5. **Fixed job title effects:** The job title contribution (J_i) is derived from a predefined average salary list.
6. **Ordered factor levels:** Categorical variables are encoded as ordered numeric values.

```
library(tidyverse)
```

Warning: package 'tidyverse' was built under R version 4.4.3

Warning: package 'tidyr' was built under R version 4.4.2

Warning: package 'dplyr' was built under R version 4.4.3

Warning: package 'forcats' was built under R version 4.4.3

Warning: package 'lubridate' was built under R version 4.4.3

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.1.4      v readr      2.1.5
v forcats    1.0.0      v stringr    1.5.1
v ggplot2    3.5.1      v tibble     3.2.1
v lubridate  1.9.4      v tidyr      1.3.1
v purrr      1.0.2

-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(dplyr)

simulate_salary_data <- function(n = 500, seed = 123) {
  set.seed(seed)

  # Simulate categorical variables
  experience_level <- factor(
    sample(c("EN", "MI", "SE", "EX"), size = n, replace = TRUE),
    levels = c("EN", "MI", "SE", "EX")
  )

  company_size <- factor(
    sample(c("S", "M", "L"), size = n, replace = TRUE),
    levels = c("S", "M", "L")
  )

  employment_type <- factor(
    sample(c("FL", "CT", "PT", "FT"), size = n, replace = TRUE),
    levels = c("FL", "CT", "PT", "FT")
  )

  countries <- c("US", "IN", "GB", "DE", "CA", "FR", "AU", "BR", "JP", "ES", "NL", "SG", "CH")
  employee_residence <- sample(countries, size = n, replace = TRUE)
  company_location <- sample(countries, size = n, replace = TRUE)

  remote_ratio <- sample(c(0, 50, 100), size = n, replace = TRUE)

  # Job title and salary bonuses
  job_title_bonus <- c(
    "Research Engineer" = 210600,
    "Machine Learning Engineer" = 197709,
```

```

"Research Scientist" = 196069,
"Software Engineer" = 190772,
"Product Manager" = 190560,
"Applied Scientist" = 181093,
"AI Engineer" = 172847,
"Engineer" = 171732,
"Manager" = 168842,
>Data Architect" = 160391,
>Data Scientist" = 156993,
"Analytics Engineer" = 156634,
>Data Engineer" = 147826,
"Business Intelligence Engineer" = 132704,
"Associate" = 132616,
"Consultant" = 123738,
"Analyst" = 113337,
>Data Analyst" = 105580,
>Data Manager" = 102056,
>Data Specialist" = 81343
)

job_titles <- names(job_title_bonus)
dominant_titles <- c("Data Scientist", "Data Engineer", "Software Engineer", "Data Analyst")
non_dominant_titles <- setdiff(job_titles, dominant_titles)

dominant_prob <- rep(0.50 / length(dominant_titles), length(dominant_titles))
non_dominant_prob <- rep(0.50 / length(non_dominant_titles), length(non_dominant_titles))

job_title_probs <- c(dominant_prob, non_dominant_prob)
names(job_title_probs) <- c(dominant_titles, non_dominant_titles)

job_title <- sample(job_titles, size = n, replace = TRUE, prob = job_title_probs)
job_title_effect <- job_title_bonus[job_title]

salary <- 0.7 * job_title_effect +
  3000 * as.numeric(experience_level) +
  1500 * as.numeric(company_size) +
  1000 * as.numeric(employment_type) +
  500 * (remote_ratio / 100) +
  rnorm(n, mean = 0, sd = 10000)

data.frame(
  experience_level,

```

```

    company_size,
    employment_type,
    employee_residence,
    company_location,
    job_title,
    remote_ratio,
    salary
)
}

```

3. Estimates

In this simulation, I am estimating the expected salary as a function of various predictors such as job title, experience level, company size, employment type, and remote work ratio. The primary response variable is the salary of each individual.

To estimate the performance of the model, I will be using the coefficient of determination (R^2).

Specifically, I will simulate a fixed number of independent datasets from the same data-generating process, fit the model to each dataset, and compute the RMSE value for each.

Let ($\hat{y}_i$) be the predicted salary and (y_i) be the actual salary for observation (i). For each simulation, the RMSE is calculated as:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2}$$

Let ($\text{RMSE}_1, \text{RMSE}_2, \dots, \text{RMSE}_k$) represent the RMSE values from each of the (k) simulations. Then I estimate the following quantities:

- **Expected RMSE:**

$$\mathbb{E}[\text{RMSE}] \approx \frac{1}{k} \sum_{j=1}^k \text{RMSE}_j$$

- **Variance of RMSE:**

$$\text{Var}(\text{RMSE}) \approx \frac{1}{k-1} \sum_{j=1}^k (\text{RMSE}_j - \overline{\text{RMSE}})^2$$

where ($\overline{\text{RMSE}}$) is the sample mean of the RMSE values.

These estimates help assess the average predictive accuracy of the model and how much its performance varies due to randomness in the data.

4. Methods

I am evaluating the performance of the LightGBM algorithm for predicting salary based on several covariates, using synthetically generated data. The prediction model is a gradient-boosted decision tree implemented via LightGBM. The model takes features such as company size, employment type, remote ratio, job title, company location, and employee residence.

To assess model performance under different tuning strategies, I will compare LightGBM across a grid of hyperparameters:

- `num_leaves` – maximum number of leaves in one tree
- `learning_rate` – step size shrinkage used to prevent overfitting
- `num_iterations` – number of boosting rounds
- `feature_fraction` – fraction of features used per iteration
- `bagging_fraction` – fraction of data used for each iteration
- `bagging_freq` – how often bagging is performed

My hypothesis:

- Higher `num_leaves` may improve training accuracy but increase overfitting risk.
- Lower `learning_rate` should stabilize training but require more iterations.
- Tuning `feature_fraction` and `bagging_fraction` should help reduce variance.

5. Performance Criteria

To assess the performance of LightGBM models under different hyperparameter settings, I use the following evaluation metric computed across all Monte Carlo repetitions:

- **Root Mean Squared Error (RMSE):**
Represents the square root of the average squared prediction error for a single simulation. For predictions ($\hat{y}_j$) and actual values (y_j), over (k) observations:

$$\text{RMSE} = \sqrt{\frac{1}{k} \sum_{j=1}^m (\hat{y}_j - y_j)^2}$$

6.Simulation Plan

Number of Simulations

- 10 simulation repetitions
- In each repetition, a synthetic dataset will be generated using the data-generating model described earlier.
- The data will be split into training (80%) and testing (20%) sets.

Parameter Settings

I will test multiple combinations of LightGBM hyperparameters.

- **num_leaves**: [15, 31]
- **learning_rate**: [0.01, 0.05]
- **num_iterations**: [100, 200]
- **feature_fraction**: [0.7, 0.8, 0.9]
- **bagging_fraction**: [0.6, 0.8, 1.0]
- **bagging_freq**: [1, 5]

This results in a grid of parameter combinations, each of which will be evaluated across all simulations.

Preprocessing

- Target encoding will be applied to categorical variables using only the training data within each simulation.
- All other features will be used as generated.

What Will Be Recorded

For each simulation and parameter setting, I will record:

- RMSE score on the test set.
- Additional statistics across simulations:
 - Mean and variance of RMSE
 - Mean and variance of MSE
- The best-performing parameter combination based on average RMSE.

```
# Target Encoding function
target_encode <- function(data, cat_col, target_col) {
  # Calculate the mean salary for each category
  encoded_values <- data %>%
    group_by_at(cat_col) %>%
    summarise(encoding = mean(!sym(target_col), na.rm = TRUE)) %>% #for each category, calculate mean
    ungroup() #!!sym() convert column name to symbol

  # Merge encoded values back into the original dataset
  data <- data %>%
    left_join(encoded_values, by = setNames(cat_col, cat_col)) %>%
```

```

    mutate(!!sym(cat_col) := encoding) %>%
    select(-encoding) # Remove the encoding column

return(data)
}

```

```

grid <- expand.grid(
  num_leaves = c(15, 31),
  learning_rate = c(0.01, 0.05),
  num_iterations = c(100, 200),
  feature_fraction = c(0.7, 0.8, 0.9),
  bagging_fraction = c(0.6, 0.8, 1.0),
  bagging_freq = c(1, 5)
)

```

```
library(lightgbm)
```

Warning: package 'lightgbm' was built under R version 4.4.3

```

simulation_results <- list()
for (sim in 1:10) {
  message(paste("Simulation", sim))

  #generate new dataset
  sim_data <- simulate_salary_data()

  # Split data
  set.seed(sim)
  idx <- sample(1:nrow(sim_data), 0.8 * nrow(sim_data))
  train_set <- sim_data[idx, ]
  test_set <- sim_data[-idx, ]

  #Target encode on train, apply same to test
  for (col in c("experience_level", "company_size", "employment_type", "employee_residence",
    train_set <- target_encode(train_set, col, "salary")
    test_set <- target_encode(test_set, col, "salary")
  }

  x_train <- as.matrix(train_set %>% select(-salary))
  y_train <- train_set$salary
  x_test <- as.matrix(test_set %>% select(-salary))
}

```



```

y_test <- test_set$salary

#run each parameter combination
for (i in 1:nrow(grid)) {
  params <- as.list(grid[i, ])
  params$objective <- "regression"
  params$metric <- "rmse"

  dtrain <- lgb.Dataset(x_train, label = y_train)

  #train lgbm model
  model <- lgb.train(
    params = params,
    data = dtrain,
    nrounds = params$num_iterations,
    verbose = -1)
  #use model for testing
  preds <- predict(model, x_test)
  #calculate rmse
  rmse <- sqrt(mean((y_test - preds)^2))

  simulation_results[[length(simulation_results) + 1]] <- cbind(grid[i, ], simulation = sim
}
}

```

Simulation 1

Simulation 2

Simulation 3

Simulation 4

Simulation 5

Simulation 6

Simulation 7

Simulation 8

Simulation 9

Simulation 10

```
# Combine results into a single data frame
results_df <- bind_rows(simulation_results)
```

```
#summarize results for each hyperparameter combo
summary_results <- results_df %>%
  group_by(num_leaves, learning_rate, num_iterations,
           feature_fraction, bagging_fraction, bagging_freq) %>%
  summarise(mean_rmse = mean(rmse), .groups = "drop") %>%
  arrange(mean_rmse)

head(summary_results)
```

A tibble: 6 x 7

	num_leaves	learning_rate	num_iterations	feature_fraction	bagging_fraction
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	15	0.05	100	0.8	0.6
2	15	0.05	100	0.9	0.6
3	31	0.05	100	0.8	0.6
4	31	0.05	100	0.9	0.6
5	15	0.05	100	0.7	0.6
6	31	0.05	100	0.7	0.6

i 2 more variables: bagging_freq <dbl>, mean_rmse <dbl>

7.Challenges/Limitations

Runtime Concerns

- **Computational Cost:** Running 10 simulations across a grid of LightGBM hyperparameter combinations can be computationally expensive.
- **Encoding Overhead:** Target encoding adds an additional preprocessing step that must be repeated in each simulation run, increasing total runtime.

Model Assumptions

- **Synthetic Data Simplifications:** The data-generating model may not fully capture all complexities of real-world data.

- **LightGBM Assumptions:** LightGBM assumes that features are independent and additive to some degree. It may not perform as well if strong interactions or feature dependencies exist and are not accounted for.

Random Variability

- **Monte Carlo Error:** There will be variability in performance metrics due to randomness in data generation, train-test splits, and model fitting.
- **Encoding Variance:** Target encoding may introduce variance if the training sample is small or if categorical variables have many unique levels.

To mitigate these limitations, I will:

- Use consistent random seeds for reproducibility.
- Aggregate results across simulations to understand average performance and variability.
- Analyze and visualize the spread of model metrics